

A01. Fluxos de datos

1.	Ficha técnica.....	6
	Contexto da actividade.....	6
	Título da actividade.....	7
	Resultados de aprendizaxe do currículo.....	7
	Obxectivos didácticos e título e descrición da actividade.....	7
	Criterios de avaliación.....	7
	Contidos.....	7
	Actividades de ensino e aprendizaxe e de avaliación, métodos, recursos e instrumentos de avaliación (exemplo).....	8
2.	A01. Fluxos de datos.....	9
2.1	Introdución.....	9
2.2	Actividade.....	9
2.2.1	Definición de fluxo ou stream.....	9
2.2.2	Tipos de fluxos e clases.....	10
	Fluxos de bytes.....	10
	Fluxos de caracteres.....	11
2.2.3	Uso dos fluxos.....	11
2.2.4	Aplicación de formatos na saída.....	12
	Sintaxe da cadea con formato.....	13
	Bandeiras.....	13
	Conversións.....	14
2.3	Tarefas.....	16
2.3.1	Tarefa 1. Nomenclatura de clases de I/O.....	16
	Enunciado.....	16
	Solución.....	16
2.3.2	Tarefa 2. Información de saída con formato.....	17
	Enunciado.....	17
	Solución.....	18
3.	Materiais.....	19
3.1	Documentos de apoio ou referencia.....	19
3.2	Recursos didácticos.....	19
4.	Avaliación.....	20
4.1	Modelo de proba para CA5.1.2 e CA5.2.....	20
	OU.1 Táboa de indicadores sobre un proxecto Java onde se utilice a consola para a realización de operacións avanzadas de saída de información.....	20
	OU. 2 Táboa de indicadores sobre un proxecto Java onde se apliquen formatos na visualización de información.....	20
	Enunciado.....	20
	Solución.....	21
	Exemplo de execución.....	21
	Exemplo de táboa de indicadores para avaliar o proxecto Java (CA5.1.2.).....	21
	Exemplo de táboa de indicadores para avaliar o proxecto Java (CA5.2.).....	21
4.2	Modelo de proba para CA5.3.....	22

<u>PE. 1 Pregunta curta sobre as posibilidades de entrada e saída da linguaxe, e as librarías asociadas.....</u>	<u>22</u>
<u>Enunciado.....</u>	<u>22</u>
<u>Solución.....</u>	<u>22</u>

1. A01. Fluxos de datos

1.1 Introducción

Na actividade que nos ocupa aprenderanse os seguintes conceptos e manexo de destrezas:

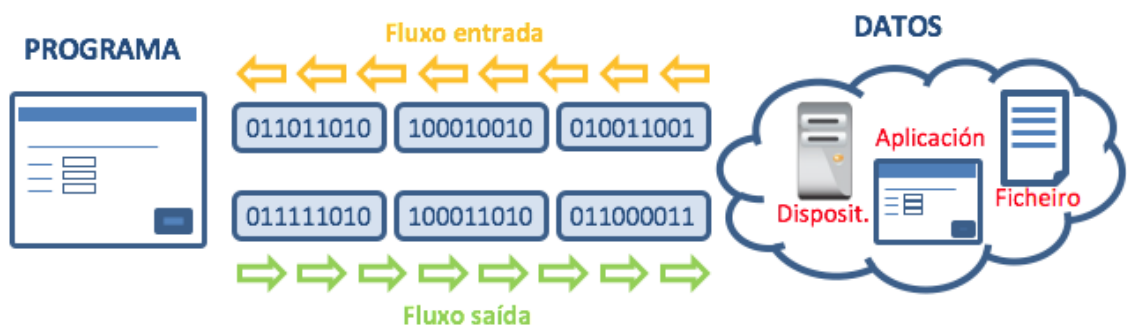
- Utilidades dos fluxos de datos.
- Tipos de fluxos de datos existentes en Java.
- As clases dispoñibles para traballar con fluxos de datos de bytes tanto de saída como de entrada.
- As clases dispoñibles para traballar con fluxos de datos de caracteres tanto de saída como de entrada.
- O procedemento a seguir para o uso de fluxos de datos.
- Os métodos que permiten dar formato á información de saída.
- A sintaxe para construír un marcador de formato imprescindible para dar formato á saída da información.

1.2 Actividade

1.2.1 Definición de fluxo ou *stream*

Os fluxos ou, como se coñece a este mesmo termo en inglés, *streams*, representan a un canal de comunicación polo que se envía unha secuencia de datos. Os fluxos poden ser de dous tipos en función do seu orixe:

- Fluxos de entrada: neste caso, a secuencia de datos ten coma orixe, por exemplo, un ficheiro ou outra aplicación e como destino a nosa aplicación. É dicir, o fluxo de datos de entrada proporciona información de entrada ao noso programa.
- Fluxo de saída: neste caso, a secuencia de datos ten coma orixe o noso programa e como destino, por exemplo, un ficheiro ou outra aplicación. Este fluxo representa a información de saída que xera o noso programa..



Na actividade 3 da unidade 1, xa vimos como utilizar un tipo de fluxo, que son os fluxos de datos estándar. Este tipo de fluxos permite obter datos de entrada ao programa desde o teclado e xerar información de saída por pantalla. Neste actividade vamos a ampliar a forma tanto de obter datos de entrada como de saída ao programa.

1.2.2 Tipos de fluxos e clases

En Java existen dous tipos de fluxos:

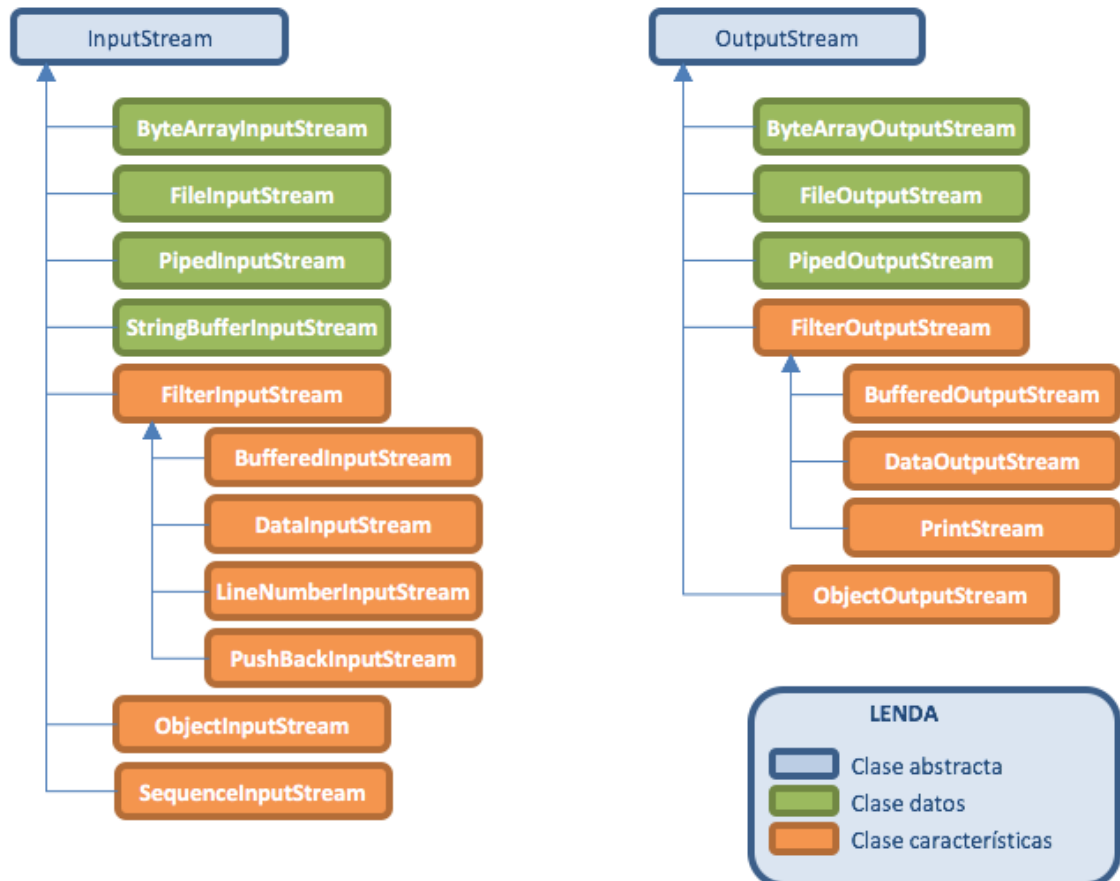
- De bytes.
- De caracteres.

Fluxos de bytes

O que se transmite a través do fluxo son bytes. Isto vai permitir poder recibir ou enviar calquera tipo de dato. Como contrapartida ten que é unha forma de procesar a entrada e saída a un nivel máis baixo, polo que se debe evitar e utilizar só para casos específicos.

Todas as clases relacionadas os fluxos de bytes herdan de dúas clases abstractas:

- `InputStream`: clase abstracta da que herdan todas as clases relacionadas con fluxos de bytes que son de lectura.
- `OutputStream`: clase abstracta da que herdan todas as clases relacionadas con fluxos de bytes que son de escritura.

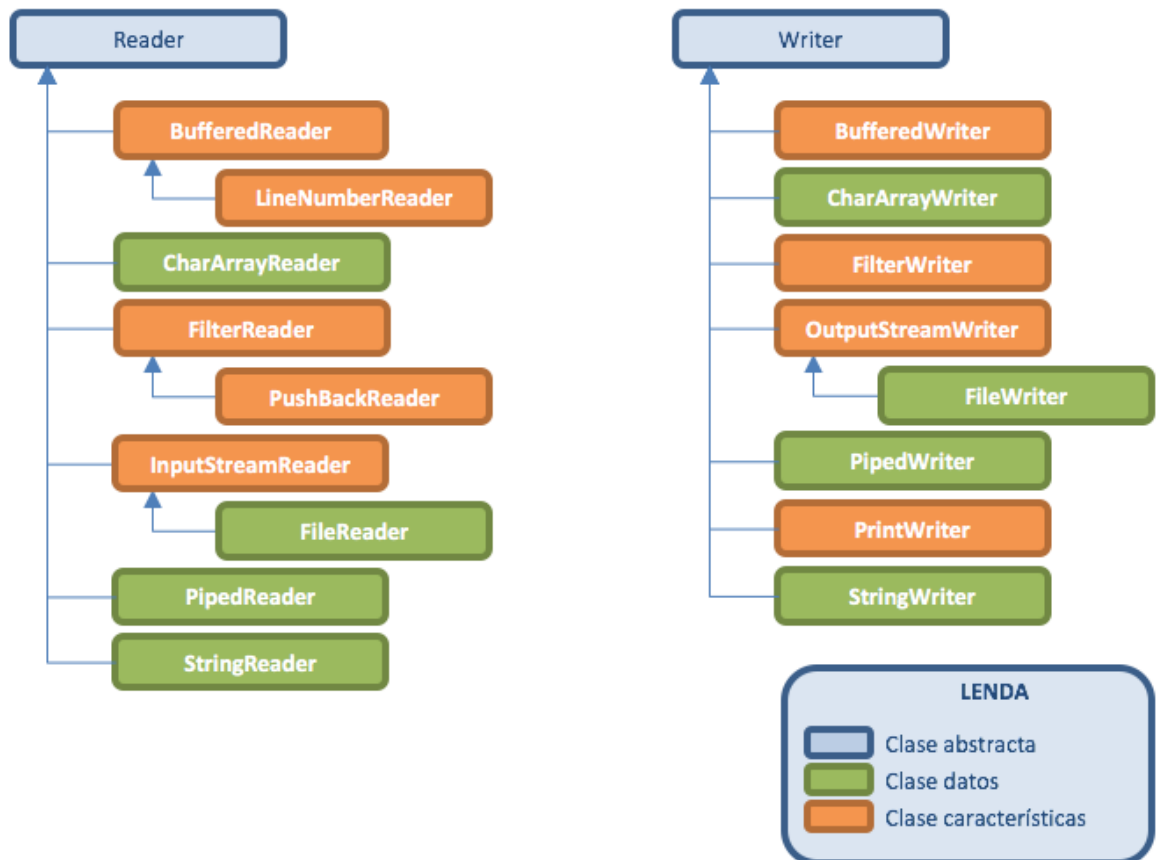


Fluxos de caracteres

O que se transmite a través do fluxo son caracteres. É o tipo de fluxo adecuado a usar, por exemplo, cando queremos ler texto almacenado nun arquivo.

Todas as clases relacionadas os fluxos de bytes herdan de dúas clases abstractas:

- **Reader**: clase abstracta da que herdan todas as clases relacionadas con fluxos de caracteres que son de lectura.
- **Writer**: clase abstracta da que herdan todas as clases relacionadas con fluxos de caracteres que son de escritura.



Todas as clases relacionadas coa entrada e saída en Java atópanse no paquete `java.io`.



Realizar a Tarefa 1 “Nomenclatura de clases de I/O”, onde o alumnado deberá procurar información da nomenclatura e significado do nome das clases de entrada e saída.

1.2.3 Uso dos fluxos

Para usar os fluxos hai un procedemento xenérico a seguir:

- **Abrir o fluxo**: utilizaranse para elo as clases que indican a orixe ou o destino dos datos. É o paso que establece a conexión co dispositivo do que se vai a ler, ou onde se van enviar os datos.

- Caracterizar o fluxo: se lle engaden as características ao fluxo que establecerá a forma de traballar con el. Por exemplo se pode indicar para un fluxo creado que vai a utilizar un buffer.
- Ler ou escribir datos mentres haxa información para ler desde o orixe ou datos a enviar ao destino.
- Pechar o fluxo. Unha característica dos fluxos que se viron na unidade 1, é dicir, dos fluxos estándar, é que non son necesario pechalos. Pola contra, se creamos un fluxo para ler dun ficheiro é necesario pechar ese fluxo. O efecto de non pechar o fluxo dun ficheiro é o mesmo que se temos un programa que está abrindo ese ficheiro e non o pecha, ningún outro programa pode acceder ao mesmo ficheiro para modificalo.

Vexamos un exemplo onde utilizamos e combinamos dúas das clases vistas anteriormente, para crear un fluxo e asociarlle unha característica.

Exemplo de creación e caracterización dun fluxo

```
FileReader file = new FileReader("exemplo.txt");
BufferedReader br = new BufferedReader(file);
```

A primeira liña deste exemplo establece un fluxo de lectura desde o arquivo con nome “exemplo.txt”. A segunda liña, establece que para a lectura deste fluxo vaise a utilizar un buffer, co que se van a facer menos operacións de entrada e saída no ficheiro.

1.2.4 Aplicación de formatos na saída

As clases `PrintStream` e `PrintWriter` implementan un conxunto de métodos para escribir tanto bytes como caracteres. Adicionalmente, estas clases permiten o formato dos obxectos.

Ata agora utilizamos os métodos `print()` e `println()` mediante unha chamada desde o fluxo estándar `System.out`, que é un obxecto da clase `PrintStream`. O único que podemos facer con eles foi imprimir os valores á maneira estándar. Por exemplo, a impresión dun dato de tipo `double` resultaba ser o número cunha unha gran cantidade de números decimais. Con estes dous métodos non hai maneira de aplicar formato á saída.

Estas clases proporcionan un método adicional para poder aplicar un formato á información de saída. A sintaxe do método é a seguinte:

Sintaxe método `format()`

```
public PrintWriter format(String formato, Object... argumentos)
```

onde:

- `formato`: é unha cadea de caracteres que ten unha sintaxe moi concreta onde se define o formato que van a ter os datos que se impriman.
- `argumentos`: son unha lista de argumentos separados por comas aos que se fai referencia no parámetro `formato`. En caso de que haxa máis argumentos que aos que se fai referencia en `formato`, estes serán ignorados.

Vexamos entón o máis importante deste método que é como xerar a cadea de texto na que se indica o formato de saída.

Sintaxe da cadea con formato

A cadea que especifica o formato será unha cadea de texto normal que a maiores leva incrustados unha serie de marcadores de formato que indican como será o formato de saída.

O formato xenérico destes modificadores é o seguinte:

Formato modificadores

```
//Para caracteres e números
%[indiceArgumento$][bandeiras][ancho][.precision]conversion
//Para datas e horas
%[indiceArgumento$][bandeiras][ancho]conversion
```

onde:

- **indiceArgumento:** é opcional e indica a posición do argumento a formatar dentro da lista de argumentos. Para facer referencia ao primeiro argumento da lista úsase “1\$”, para o segundo “2\$”, e así sucesivamente.
- **bandeiras:** é tamén opcional. Consiste nun conxunto de caracteres que modifican o formato de saída. Este conxunto depende do que haxa en `conversion`.
- **ancho:** tamén opcional. É un número enteiro positivo que indica o número mínimo de caracteres que serán escritos na saída.
- **precision:** tamén opcional. É un enteiro non negativo que normalmente se usa para restrinxir o número de caracteres na saída. O seu comportamento adáptase ao que haxa en `conversion`.
- **conversion:**



Enlace á información completa da sintaxe da cadea con formato de Java: <https://docs.oracle.com/javase/8/docs/api/java/util/Formatter.html#syntax>

Bandeiras

A continuación móstrase as bandeiras posibles:

Carácter	Descrición
-	O resultado sairá xustificado.
#	Formatea nunha forma alternativa para o marcador de formato.
+	O resultado sempre incluírá o signo.
espazo	O resultado incluírá un espazo para valores positivos.
0	O resultado cubrírase con 0 ata a lonxitude especificada.
,	Engade un separador de grupos. Aplicado só a números.
(	O resultado pechará entre parénteses números negativos.
<	Para reusar o valor do argumento utilizado no anterior marcador de formato. Usado sobre todo no formateo de datas e horas

Conversións

A continuación móstrase algúns dos caracteres que poden ir en lugar de `conversion` no caso de texto e números:

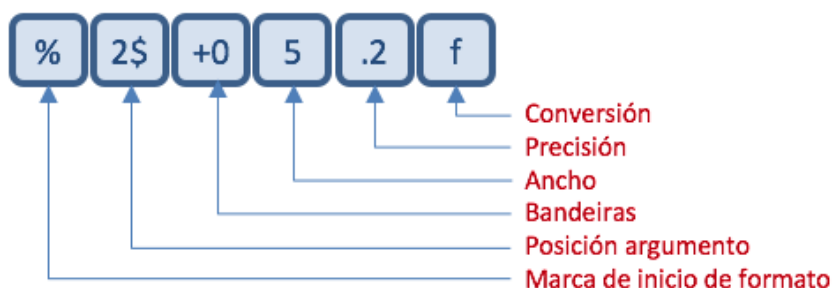
Carácter	Descrición
B ou b	Para representar a un booleano.

S ou s	Para representar a un string.
C ou c	Para representar a un carácter.
d	Para representar a un enteiro.
f	Para representar a un número decimal.
%	Para representar ao literal “%”
n	Para representar a un separador de liña.

Nesta outra táboa móstrase algúns dos valores que pode tomar o segundo carácter no caso das datas e horas:

Carácter	Descrición
H	Para representar á hora en formato de 24 horas e con dous díxitos.
I	Para representar á hora en formato de 12 horas e con dous díxitos.
M	Para representar os minutos dentro da hora, formatado con dous díxitos.
S	Para representar os segundos dentro do minuto, formatado con dous díxitos.
y	Para representar ao ano, formatado con dous díxitos.
m	Para representar ao mes, formatado con dous díxitos.
d	Para representar ao día, formatado con dous díxitos.

Na seguinte imaxe pode verse un exemplo desta sintaxe aplicada a un número decimal:



No exemplo anterior, estamos facendo referencia ao segundo argumento (2\$), que se vai a mostrar con signo (+). Da parte enteira mostrarase 5 díxitos (5) e se o número ten menos de 5 díxitos vaise encher con 0 (0), para a parte decimal só se mostrarán 2 díxitos, e a conversión é dun número flotante.

Vamos a aplicar este exemplo nun anaco de código para ver o funcionamento do método:

Exemplo `format()`

```
String s = "Ola";
float numero = 43.567756456463f;
System.out.format("O número sen formato é: %2$f e con formato é: %2$+05.2f %n", s, numero);
```

Como saída obtemos o seguinte:

```
run:
O número sen formato é: 43,567757 e con formato é: +43,57
BUILD SUCCESSFUL (total time: 0 seconds)
```

Outro método que proporcionan estas clases e que permite o formato de saída é o método `printf()`. Non comentamos o seu funcionamento, pois é exactamente igual a `format()`.

Podemos consultar o enlace indicado a continuación, onde se mostra información máis detallada do formato e no que se inclúen diversos exemplos que poden axudar a clarificar como dar formato á información de saída.



Enlace á información completa da sintaxe da cadea con formato de Java: http://www.java2s.com/Tutorials/Java/Java_Format/0050__Java_Printf_Style_Overview.htm



Realizar a Tarefa 2 “Información de saída con formato”, onde o alumnado deberá darlle o formato indicado aos datos que se listan na tarefa.

1.3 Tarefas

As tarefas propostas para esta actividade son as seguintes:

- Tarefa 1. **Nomenclatura de clases de I/O**. Nesta tarefa o alumnado deberá procurar información sobre como se constrúen o nome das clases relacionadas coa entrada e saída de datos.
- Tarefa 2. **Información de saída con formato**. Nesta tarefa o alumnado deberá darlle os formatos que se indican a unha serie de datos para mostralos por pantalla.

1.3.1 Tarefa 1. Nomenclatura de clases de I/O

Enunciado

Os nomes das clases relacionadas coa entrada e saída de datos de Java segue unha sintaxe moi concreta e utiliza uns nomes específicos que permiten coñecer o funcionamento xeral da clase.

Procura información en internet e indica a que se adica cada clase e en que te fundamentas.

Solución

Nas imaxes aportadas na actividade, nas que se representan as xerarquías de clases utilizadas para a entrada e saída de datos en Java, as clases aparecen en dúas cores.

- Verde: representa ás clases que indican a orixe ou o destino dos datos, é dicir, ao dispositivo ao que se conecta o fluxo, ben para ler datos del ou para escribir nel os datos xerados polo programa.
- Laranxa: representa ás clases que engaden particularidades á forma na que se envían os datos.

Podemos dicir a nivel xeral que o nome das clases está composto por dúas partes:

- Primeira parte: pode indicar dúas cousas, ou ben cal é a orixe ou destino do fluxo ou ben, que característica modifica o comportamento do fluxo de datos.
Cando se quere indicar a orixe ou o destino do fluxo, o nome da clase ten como primeira parte as seguintes opcións:
 - `ByteArray`: permite crear un fluxo de bytes sendo a orixe/destino un array de bytes.
 - `CharArray`: permite crear un fluxo de bytes sendo a orixe/destino un array de caracteres.
 - `File`: especifica que a orixe ou o destino do fluxo é un arquivo de disco.
 - `Piped`: permite a comunicación entre dous fíos, de maneira que a saída xerada nun dos fíos é a entrada de datos que utiliza o outro fío.
 - `String`: permite crear un fluxo de bytes sendo a orixe/destino unha cadea de caracteres.

- StringBuffer: (esta clase está obsoleta) permite crear un fluxo de bytes sendo a orixe unha cadea de caracteres.
Cando se quere indicar unha características determinada, o nome da clase ten como primeira parte as seguintes opcións:
- Buffered: indica que se vai a utilizar un buffer para as lecturas e escrituras. Desta forma se reducen as operacións de lectura e escritura sobre o dispositivo.
- Data: permite traballar cos datos primitivos de Java de maneira que sexan independente da plataforma.
- Filter: permite aplicar filtros ao fluxo de datos.
- InputStream: permite facer unha conversión dun fluxo de entrada que utiliza bytes, noutro que utiliza caracteres.
- LineNumber: teñen en conta os números de liñas.
- Object: permite a serialización de obxectos.
- OutputStream: permite facer unha conversión dun fluxo de saída que utiliza bytes, noutro que utiliza caracteres.
- Print: permite imprimir en formato de texto os obxectos de Java.
- PushBack: permite o retroceso.
- Sequence: representa unha concatenación lóxica de outros fluxos de entrada.
- Segunda parte: de que tipo de fluxo se trata, entrada ou saída, e se é de bytes ou caracteres.
 - InputStream: indica que se trata dun fluxo de bytes de entrada.
 - OutputStream: indica que se trata dun fluxo de bytes de saída.
 - Reader: indica que se trata dun fluxo de caracteres de entrada.
 - Writer: indica que se trata dun fluxo de caracteres de saída.

1.3.2 Tarefa 2. Información de saída con formato

Enunciado

Crear un pequeno programa no que se declaren 4 variables dos seguintes tipos:

- Enteiro con valor negativo
- Float con polo menos 3 cifras na parte enteira e 8 na parte decimal.
- Cadea.
- Data e hora.

A cada unha destas variables asignaráselle un valor e a continuación deberase mostrar o seu valor por pantalla co formato establecido a continuación. Cada apartado corresponde cunha única instrución de formatado:

- A variable enteira mostrarase por un lado con signo, e a continuación con parénteses.

- A variable float mostrarase en notación real con punto fixo, e tamén en notación científica.
- A variable float de maneira que se mostren 10 díxitos con 3 para a parte decimal, e en caso de que o número de díxitos sexa menor encheranse os ocos con 0, tamén se mostrará o mesmo formato pero con aliñación esquerda.
- Mostrarase o valor da variable float e o da variable enteira e a continuación o resultado da súa división, mostrarase a continuación de novo os dous valores e o resultado da súa suma.
- A variable de tipo cadea en maiúscula, a variable de tipo cadea ocupando 15 espazos, e a cadea con 15 espazos pero aliñación á esquerda.
- A variable de tipo data de maneira que se vexa un texto coma o seguinte: “Son as 13 horas 50 minutos 55 segundos (formato PM) do 21 de outubro do ano 2017”. O PM non será literal, senón que será parte do formato. Se a hora fora as 01 mostraría automaticamente AM.

Solución

A continuación móstrase o código correspondente:

```
int valor1 = -59;
float valor2 = 456.645545345f;
String valor4 = "Ola Mundo!";
LocalDateTime data = LocalDateTime.now();
System.out.printf("Valor entero con signo, **%1$d**, con paréntesis **%1$(d** %n", valor1);
System.out.printf("Notación real: **%1$f**, notación científica: **%1$e** %n", valor2);
System.out.printf("Notación real: **%1$010.3f**, notación científica: **%1$-10.3f** %n", valor2);
System.out.printf("División de **%1$f** e **%2$d** = **%3$f** -- Suma de %1$f e %2$d = %4$f %n", valor2, valor1, valor2 / valor1, valor2 + valor1);
System.out.printf("Cadea en maiúscula: **%1$S** , ocupando 15 espazos: **%1$15S** , aliñación esquerda: **%1$-15S** %n", valor4);
System.out.printf("Son as %tH horas %tM minutos %tS segundos (formato %Tp) do %te de %tB do ano %tY%n", data);
```

Na seguinte captura pode verse parte do resultado da execución:

```
Valor entero con signo, **-59**, con paréntesis **(59)**
Notación real: **456,645538**, notación científica: **4,566455e+02**
Notación real: **000456,646**, notación científica: **456,646 **
División de **456,645538** e **-59** = **-7,739755** -- Suma de 456,645538 e -59 = 397,645538
Cadea en maiúscula: **OLA MUNDO!** , ocupando 15 espazos: **   OLA MUNDO!** , aliñación esquerda: **OLA MUNDO! **
Son as 13 horas 50 minutos 55 segundos (formato PM) do 21 de octubre do ano 2017
```

2. Autoavaliación

Enunciado

Crear un pequeno programa no que se declaren 4 variables dos seguintes tipos:

- Enteiro con 5 díxitos.
- Float con polo menos 3 cifras na parte enteira e 8 na parte decimal en negativo.
- Cadea.
- Data e hora.

A cada unha destas variables asignaráselle un valor e a continuación deberase mostrar o seu valor por pantalla co formato establecido a continuación. Cada formato indicado a continuación, corresponde cunha única instrución de formatado:

- Formato 1: mostrarase o valor correspondente da variable enteira en octal sen ningún marcador do sistema de numeración utilizado, e tamén o valor en octal pero co marcador do sistema octal.
- Formato 2: mostrarase o valor correspondente da variable enteira en hexadecimal. O marcador do sistema de numeración estará en maiúscula.
- Formato 3: mostrarase o valor da variable enteira co separador de grupos (separador de miles).
- Formato 4: mostrarase se é verdadeiro ou falso que a variable enteira é maior que a variable `float`.
- Formato 5: mostrarase o valor da variable `float` con parénteses, se o valor é negativo, como moito mostraranse 7 díxitos e só un deles decimal.
- Formato 6: mostrarase o valor da variable de tipo cadea ocupando 20 espazos.
- Formato 7: mostrarase o valor da variable de data e hora de maneira que o texto mostrado quede como segue: “Pasan X minutos das Y horas e estamos a Z”. Onde X serán os minutos da hora, Y as horas cun só dígito e Z será o nome do día da semana correspondente á data.

Enunciado

Na seguinte táboa indícanse unha serie de tipos de fluxos e características a aplicar sobre os fluxos. Deberase cubrir a táboa de maneira que se indique cal é a clase máis adecuada para representar ao fluxo ou á característica.

Fluxo ou característica	Clase
▪ Buffer que se vai a utilizar para a escritura dun fluxo de caracteres.	▪
▪ Fluxo de datos que permite a comunicación entre dous fíos. Para recibir a información do outro fío.	▪

■ Permite imprimir en formato de texto os obxectos de Java cun fluxo de bytes.	■
■ Fluxo de caracteres onde a orixe é un ficheiro.	■
■ Buffer que se vai a utilizar para a lectura dun fluxo de bytes.	■
■ Fluxo de bytes sendo a orixe un array de bytes.	■
■ Fluxo para a serialización de obxectos.	■
■ Fluxo de bytes sendo a orixe unha cadea.	■
■ Fluxo de bytes sendo o destino un ficheiro.	■
■ Fluxo de caracteres onde o destino é un array de caracteres.	■